SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

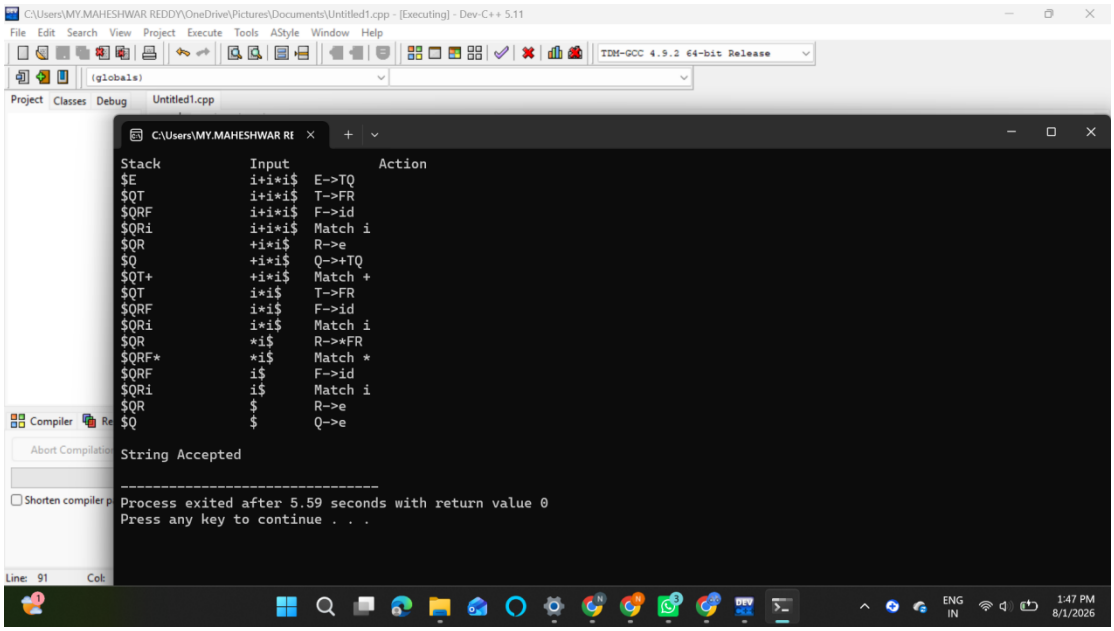
I ____M.Niharika(192365019)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Niharika

Course Faculty

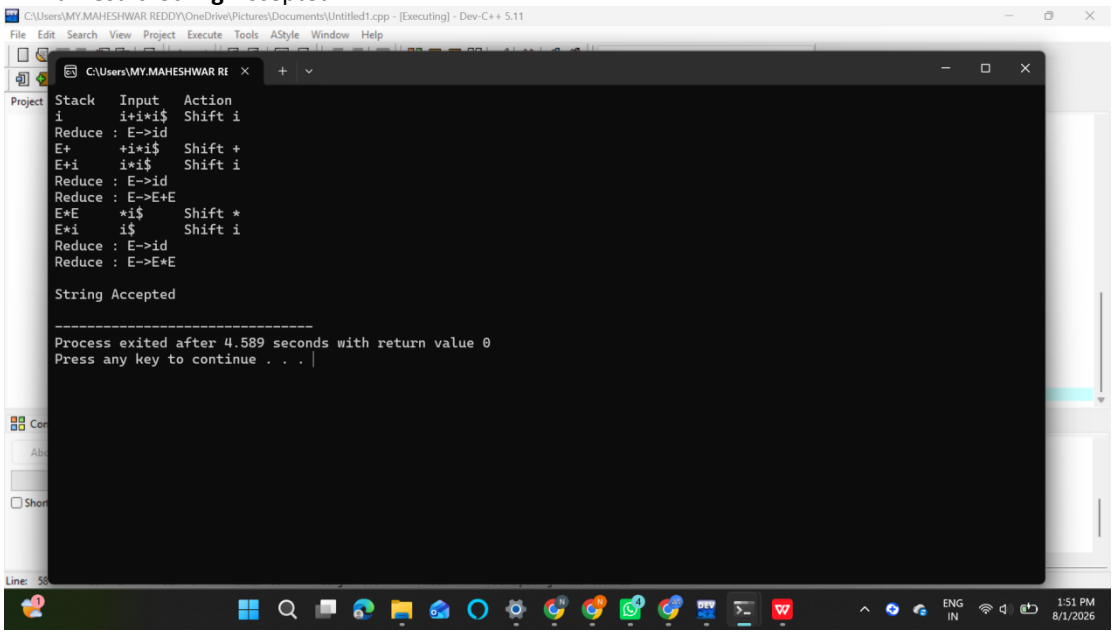
Task:
Q1)Construct an LL(1) predictive parsing table for the given grammar and demonstrate its application by parsing the
specified input string step-by-step, clearly showing stack operations, input symbols, and parsing actions.
.
Grammar:
E → T E'
E' → + T E' | ε

T → F T'
T' → * F T' | ε
F → (E) | id
Conditions:
□ Remove left recursion (if any)
□ Compute FIRST and FOLLOW sets
□ Construct LL(1) parsing table
Test Case:
Input String:
id + id * id
Expected Output:
□ Step-by-step parsing actions
□ Final result: String Accepted
Program:



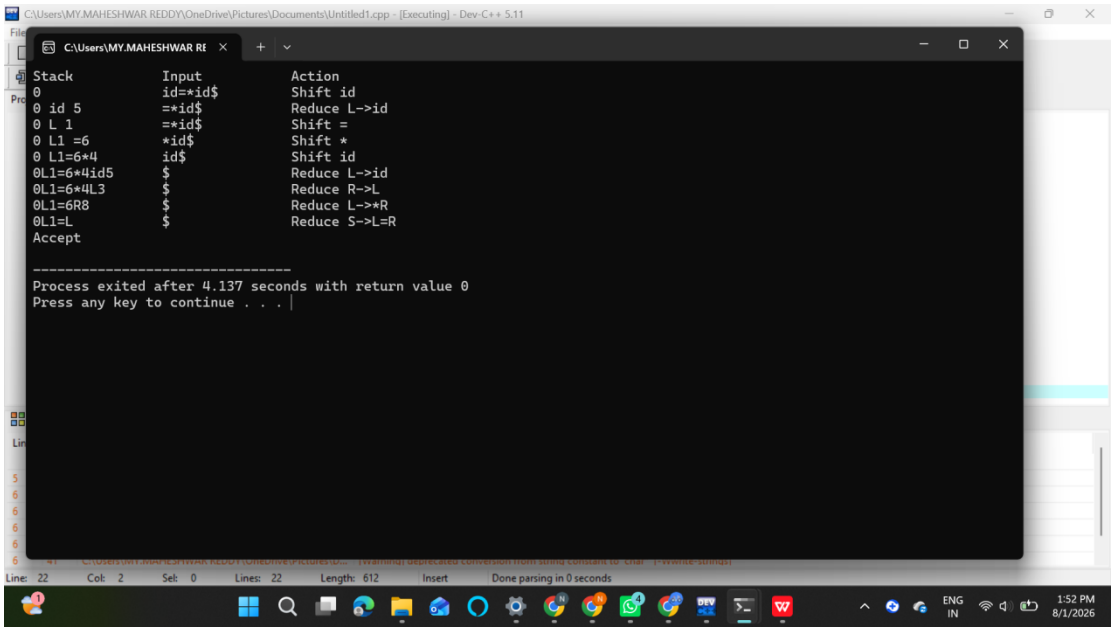
Q2. Shift-Reduce Parsing (Application Level)
Task:
Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by
showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly
indicating all reductions and decisions made during parsing.
Grammar:
E → E + E
E → E * E
E → id
Conditions:
□ Show stack, input buffer, and action at each step
□ Resolve conflicts using precedence (* has higher precedence than +*)
Test Case:
Input:
id + id * id
Expected Output:
□ Sequence of actions: Shift / Reduce / Accept

Final result: String Accepted



Q3.Construct the LR parsing tables for the given grammar and perform a detailed comparison of different LR
parsing techniques (SLR, LALR, and CLR), highlighting their construction steps, state transitions, table sizes, and
any conflicts encountered.
S → L = R | R
L → * R | id
R → L
Tasks

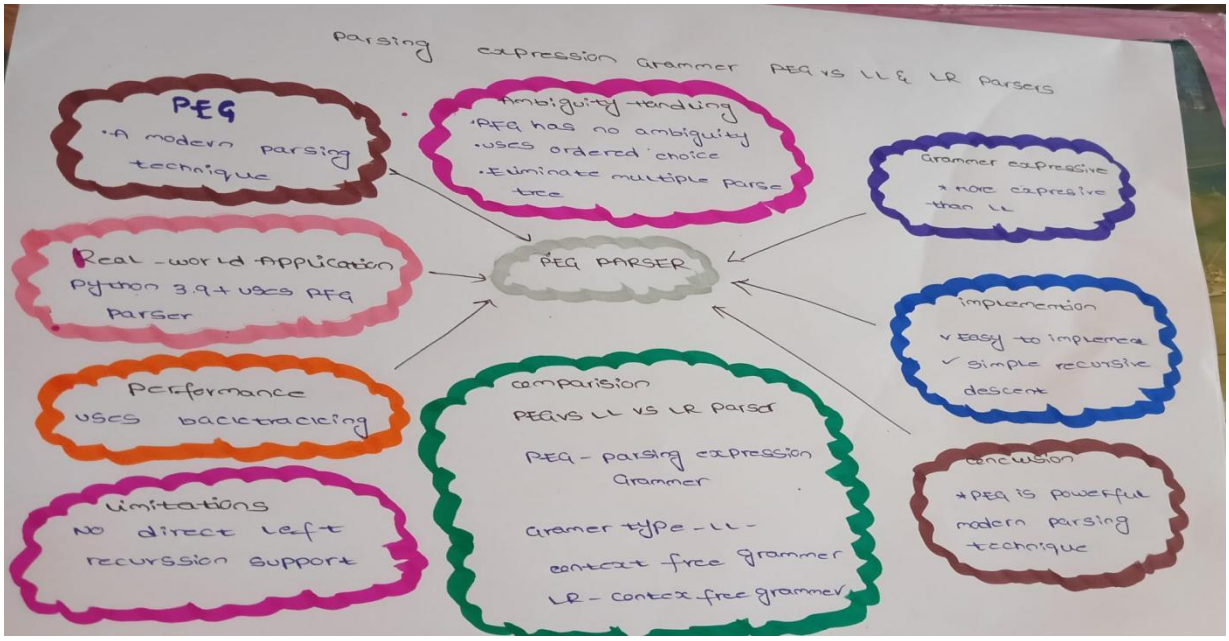
(i) CLR (Canonical LR) Design
Construct the canonical collection of LR(1) items
Build the CLR parsing table
Show all item sets and transitions
(ii) SLR Parser Design
Compute FIRST and FOLLOW sets
Construct the SLR parsing table
Identify and explain any shift-reduce or reduce-reduce conflicts
(iii) LALR Parser Design
Merge compatible LR(1) states
Construct the LALR parsing table
Compare with CLR and show how states are reduced
(iv) Parsing Demonstration
Using any one of the constructed parsing tables,
parse the input string:
id = * id
Show:
Stack
Input buffer
Action (Shift / Reduce / Accept)



Q4. Critically evaluate the effectiveness of Parsing Expression Grammars (PEG) as a modern parsing technique by comparing them with traditional LL and LR parsers in terms of ambiguity handling, grammar expressiveness, implementation complexity, performance, and real-world applicability.

Your answer should:

1. **Analyze** how PEG parsers resolve ambiguity differently from CFG-based parsers
2. **Compare** PEG with LL and LR parsing in terms of:
    - Grammar expressiveness
    - Ease of implementation
    - Performance and backtracking
3. **Examine** real-world adoption (e.g., Python's PEG parser) and justify why PEG was preferred
4. **Critically evaluate** limitations of PEG (e.g., lack of left recursion support, ordered choice behavior)
5. **Conclude** whether PEG is suitable for all programming languages or only specific cases, with justification



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation